

Building a Movie Recommendation System

Madison Berrios
Advanced Techniques in DS
Dr. Junyi Ye
1 May 2026

Introduction:

With so many choices available online, it can be difficult for users to decide what to watch, buy, or listen to. Recommendation systems help solve this problem by predicting what a user might like and suggesting items that match their preferences. These systems are used in many real-world platforms such as Netflix, Amazon, and Spotify to create a more personalized experience.

In this project, I focused on collaborative filtering, which makes recommendations based on patterns in user behavior rather than item features. This approach allows the model to learn from user interactions and identify similarities between users to generate recommendations.

Dataset & EDA:

I used the CiaoDVD dataset, which contains user ratings for movies on a scale from 1 to 5. The dataset includes information such as user IDs, movie IDs, and ratings.

From my exploratory data analysis, I found that the dataset is highly sparse. There are many possible user-movie combinations, but only a very small percentage actually have ratings. This makes the recommendation problem more challenging because there is limited information for each user.

I also observed that most ratings are high (mostly 4s and 5s), which shows a skewed distribution. This is common because users are more likely to rate movies they enjoyed.

Another important finding was the long-tail distribution, where most users rated only a few movies and most movies received only a few ratings. This affects models like KNN because it reduces overlap between users.

Models:

I implemented three different recommendation models.

The first model was the Global Average, which predicts the same average rating for every user and movie. This model does not learn any patterns but serves as a baseline to compare against.

The second model was SVD (matrix factorization). This model learns hidden patterns in the data by representing users and movies as vectors. It uses these learned patterns to predict ratings. The

idea is that users and movies can be mapped into a lower-dimensional space that captures preferences and similarities.

The third model was User-Based KNN, which finds similar users based on their rating patterns and predicts ratings using the average of their neighbors. This method relies on user similarity, which can be difficult when the dataset is very sparse.

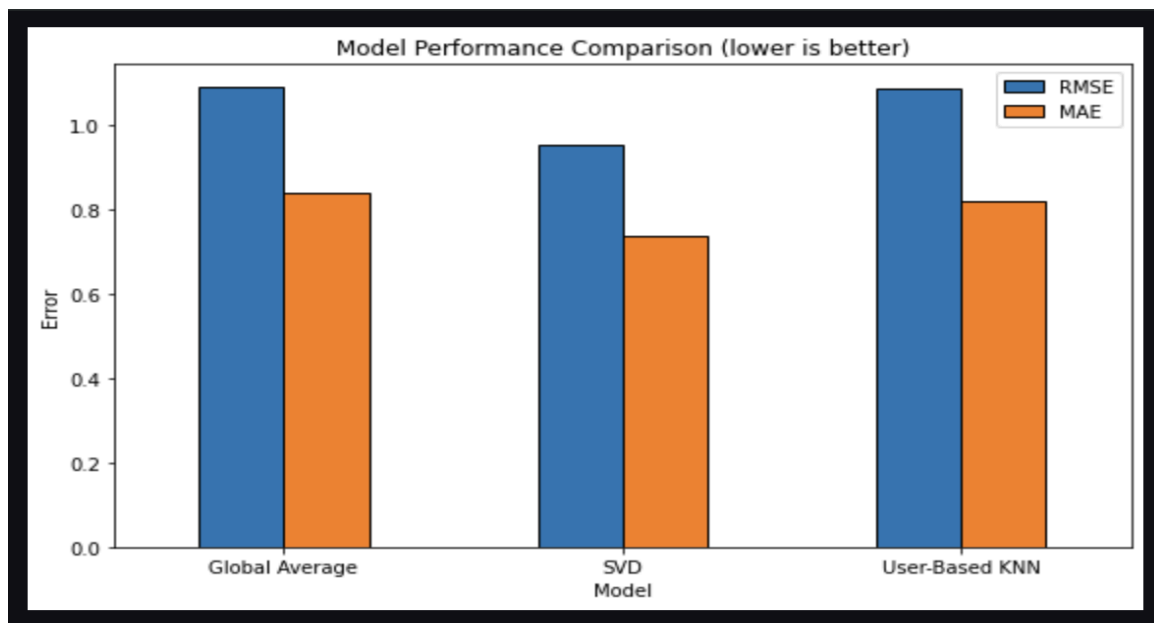
Model Evaluation Metrics:

I evaluated the models using RMSE and MAE, where lower values indicate better performance. I also used Precision and Recall to evaluate recommendation quality.

The results showed that SVD performed the best, with the lowest RMSE and MAE. This means it was the most accurate at predicting ratings.

User-Based KNN performed slightly better than the baseline but not by much, likely due to the sparsity of the dataset.

Both SVD and KNN had similar Precision and Recall scores, meaning they performed similarly when ranking top recommendations, even though SVD was better at predicting exact ratings.



Vibe Coding Reflection:

For Phase 1 of this project, I didn't use any AI. I loaded the data, did the basic statistics, created the visualizations, and handled the cleaning on my own because I was already comfortable with that part. Where AI really helped was in Phase 2, when I had to actually build the recommendation models. Since I had never worked with collaborative filtering or matrix factorization before, I used ChatGPT and Claude to help me write the code for SVD and User-Based KNN.

The most helpful part was debugging. Whenever I ran into an error, I would paste the traceback or upload a screenshot, and the AI could usually tell me what was wrong and how to fix it. That honestly saved me a lot of time. It also helped me with GitHub, since it was my first time using it. I didn't really know how to set up a repository or organize files, and at one point my files were in the wrong folder and I didn't even know why. AI helped me fix that step by step and showed me how to write better commit messages.

That being said, AI wasn't perfect. Sometimes it gave me solutions that didn't work, especially with the numpy error, and I had to try multiple fixes before something worked. I also started the project using Gemini in Google Colab, but it wasn't very helpful, so I switched to VS Code with Claude. Another issue was that different tools would give different answers for the same problem, which made it hard to know what to trust.

Overall, the biggest thing I learned is that AI is really useful for small, specific tasks, but I can't rely on it for everything. I still need to understand what my code is doing and think critically about the solutions it gives me. I think the main takeaway from this assignment was learning when to use AI and when to figure things out on my own, and I definitely feel more confident doing that now than I did at the start.